

课程笔记

斯坦福 CS336 讲座 1：概述与分词

课程笔记

2026 年 6 月



视频作者/频道：斯坦福 CS336 教学团队

发布日期：2026 年春季

视频时长：约 79 分钟

视频链接：<https://www.bilibili.com/video/BV12gLx6xEcq?p=1>

目录

1	课程介绍与背景	3
1.1	教学团队	3
1.2	可执行讲座	3
1.3	课程核心理念	4
1.3.1	三大可迁移知识类别	4
1.3.2	Bitter Lesson 的澄清	5
1.3.3	为什么小模型实验有其局限	5
1.3.4	效率：课程的核心主题	5
1.4	本章小结	5
2	语言模型发展简史	6
2.1	引言：为什么了解历史很重要	6
2.2	前神经网络时代 (Pre-neural, 2010 年前)	6
2.3	神经网络时代 (2010 年代)	7
2.3.1	LSTM 与早期神经语言模型	7
2.3.2	Seq2Seq 与注意力机制	7
2.3.3	Transformer 革命	7
2.4	预训练时代 (2010 年代末-2020 年代初)	7
2.5	现代大语言模型	7
2.6	本章小结	8
3	课程安排与教学理念	8
3.1	课程大纲	8
3.2	作业设计哲学	9
3.3	AI 辅助学习政策	9
3.4	计算资源	9
3.5	评分标准	9
3.6	本章小结	9
4	硬件与系统基础	10
4.1	GPU 架构概览	10
4.2	DGX 系统架构	11
4.3	Roofline 分析	11
4.4	内核与算子融合	12
4.5	并行化简介	12
4.6	本章小结	13
5	分词技术	13
5.1	为什么需要分词	13
5.2	分词策略的比较	14
5.3	BPE 算法详解	15
5.3.1	算法步骤	15

5.3.2	代码实现	16
5.3.3	编码性能与优化	17
5.4	各种 Tokenizer 的比较	18
5.5	分词的未来方向	18
5.6	本章小结	18
6	总结与延伸	19
6.1	讲座核心回顾	19
6.2	当前与未来的约束	19
6.3	进一步思考	20
6.4	拓展阅读	20

1 课程介绍与背景

1.1 教学团队

斯坦福 CS336 (Language Models From Scratch) 是 2026 年春季开设的一门研究生课程，目标是带领学生从零开始理解和构建大语言模型。这是该课程的第三次开设。



图 1: CS336 课程标题页与教学团队

主讲教师 **Percy Liang** 在开场时提到，自己做语言模型已有 20 年历史，”不过大部分时间做的都是小语言模型”。他坦言两年前刚开始教这门课时，自己也不太确定学生会有什么期待，但惊喜地发现”有这么多同学想学怎么从零搭建语言模型”。

联合讲师 **Tatsunori Hashimoto** (Tatsu) 负责讲授模型架构扩展性等主题。助教团队包括：

- **Marcel Rød**: 研究方向为模型架构、高阶梯度优化和模型训练
- **Herman Brunborg**: 幽默地表示”一年前我还完全不了解大语言模型是怎么回事, 当时 token 对我只是电子游戏里收集的代币”
- **Steven Cao**

1.2 可执行讲座

核心概念：可执行讲座 (Executable Lecture)

CS336 的讲座内容以 Python 程序的形式组织。当讲师”逐行运行”幻灯片时，实际上是在执行一个动态演示的 Python 程序。学生可以逐行浏览代码，看到讲座的层次结构，并且课后可以直接运行这些代码来复习和实验。

⁰ 视频画面时间区间：00:00:00–00:00:10。

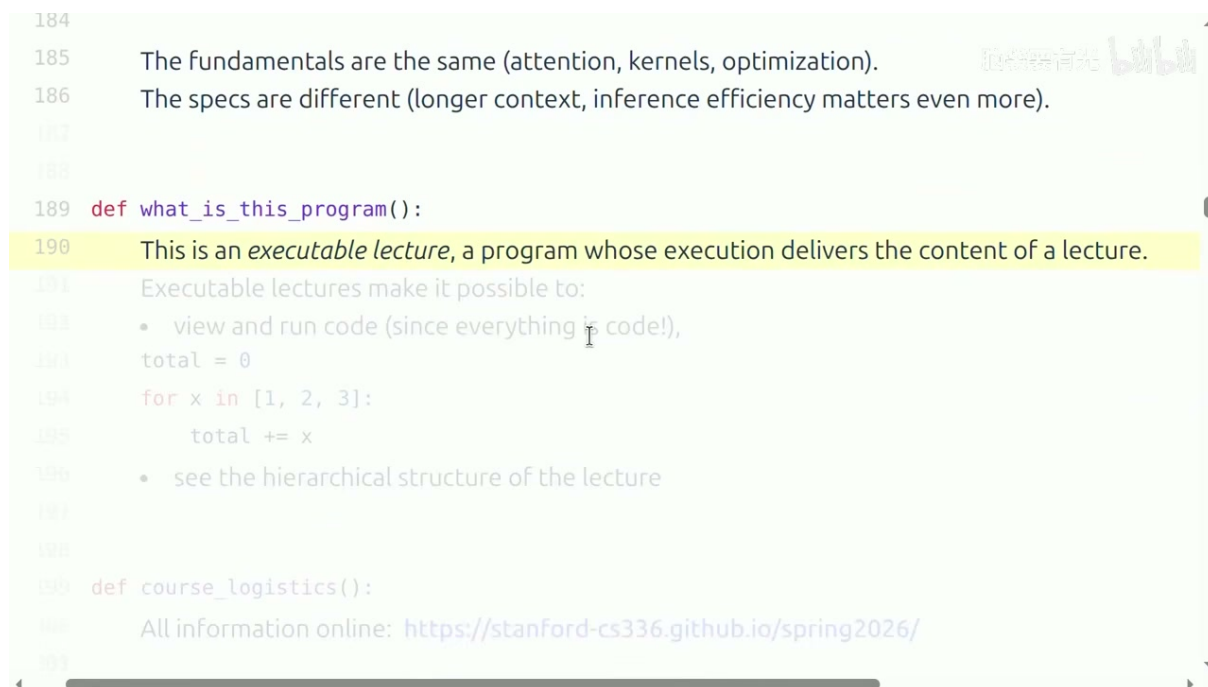


图 2: 可执行讲座的概念说明

这种形式的好处在于：

- 所有内容都是可运行的代码，消除了”幻灯片伪代码”与实际实现之间的鸿沟
- 学生可以看到讲座的层次结构（函数调用关系）
- 课后可以直接在本地环境复现和修改

1.3 课程核心理念

1.3.1 三大可迁移知识类别

Percy Liang 明确将学生能从课程中获得的知识分为三个层次：

三大可迁移知识类别

1. **机制** (Mechanism)：事物如何工作——分词器、Transformer、优化器、并行策略等每个组件的内部原理
2. **思维方式** (Mindset)：如何着手构建语言模型——包括性能分析 (profiling)、基准测试 (benchmarking)、以效率为核心的思考习惯
3. **直觉** (Intuition)：哪些建模和数据决策能带来好的效果

一个重要的警告：直觉不跨规模迁移

Percy Liang 特别警告：关于哪些建模和数据决策有效的直觉，并不一定能跨规模迁移。在小模型上有效的技巧，在大模型上未必成立。这意味着即使你在课程作业的小规模实验上积累了经验，将这些直觉直接应用到生产级大模型时仍需谨慎验证。

⁰视频画面时间区间：00:19:30–00:20:00。

1.3.2 Bitter Lesson 的澄清

讲座中，Percy Liang 直接回应了对 Sutton 著名文章《The Bitter Lesson》的一种常见误读：

对”Bitter Lesson” 的误读与正解

错误理解：”规模就是一切，算法无关紧要。”

正确理解：可扩展的算法（scalable algorithms）才重要。规模确实带来了质的飞跃，但前提是你使用的算法能够有效地利用这些规模。盲目堆叠计算而不优化算法，是一种浪费。

Percy 还引用了 SwiGLU 论文中一段令人印象深刻的结语：”我们不提供任何解释，我们将这些架构的成功全归因于上天眷顾。”（”We attribute success to divine favor.”）这句话既幽默又发人深省——它提醒我们，即使在最前沿的研究中，我们对许多设计选择的理解仍然是有限的。

1.3.3 为什么小模型实验有其局限

课程的大部分作业在小规模（相对于前沿模型）上进行，但 Percy 给出了两个具体的经验论据，说明小模型不能代表前沿模型的行为：

1. **FLOP 分布的变化：**在小模型中，MLP 层约占 44% 的总 FLOPs；而在 175B 参数规模下，这一比例上升到约 80%。这意味着优化优先级在不同规模下完全不同。
2. **涌现行为（Emergence）：**在小模型上，zero-shot 和 few-shot 学习在各种任务上”基本上都没有效果”，性能接近于随机猜测。只有在达到某个临界规模后，这些能力才会突然”涌现”。如果你只在小规模上研究，你可能永远无法观察到这些现象。

1.3.4 效率：课程的核心主题

课程的设计哲学

From Scratch（从零开始）是本课程的核心哲学。Percy Liang 强调，即使现在代码助手可以直接生成一个语言模型，理解底层原理仍然至关重要。本课程的目标不是让学生学会调用 API，而是深入理解每一个组件的工作原理。

课程还有一个贯穿始终的主题：**效率**（efficiency）。给定固定的计算资源和数据预算，如何训练出最好的模型？这个问题将贯穿系统、架构、数据、训练等所有环节。

算法效率的巨大进步

Hernandez 等（2020）的研究显示，在 2012 至 2019 年间，ImageNet 上的**算法效率提升了 44 倍**。这个数字提醒我们：除了盲目扩大规模，优化算法本身同样能带来数量级的改进。这也是本课程强调”效率”的实证基础。

1.4 本章小结

CS336 是一门以实践为导向的课程，通过可执行讲座的形式，带领学生从零开始构建大语言模型。课程团队由 Percy Liang、Tatsunori Hashimoto 领衔，配合三位经验丰富的助教。课程核心理念是”from scratch”和”efficiency”——不仅要懂原理，还要能在资源约束下做出最优决策。

2 语言模型发展简史

2.1 引言：为什么了解历史很重要

Percy Liang 在讲座中花了相当长的篇幅梳理语言模型的历史脉络。他特别强调：“现代语言模型的谱系源于神经网络架构”。了解这段历史，有助于我们理解当前大模型的设计选择，以及未来可能的发展方向。

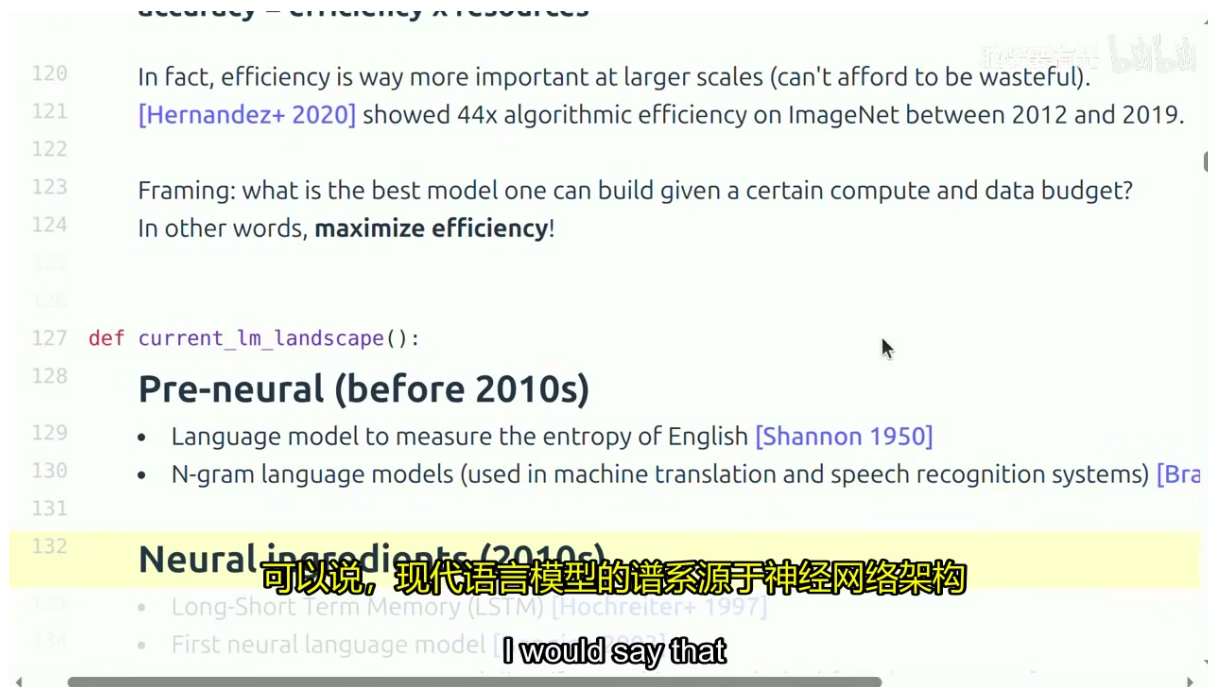


图 3: 语言模型发展史的两个阶段划分

2.2 前神经网络时代 (Pre-neural, 2010 年前)

N-gram 语言模型

N-gram 是最经典的传统语言模型。它基于马尔可夫假设：一个词的出现概率只依赖于前面 $N - 1$ 个词。N-gram 模型被广泛应用于早期的机器翻译和语音识别系统中。其局限性在于无法捕捉长距离依赖，且面临严重的数据稀疏问题。

重要的里程碑包括：

- **Shannon (1950)**：首次使用语言模型来测量英语的熵，奠定了信息论与自然语言处理的交叉基础
- **N-gram 语言模型**：在机器翻译和语音识别系统中大量使用

⁰ 视频画面时间区间：00:12:00–00:12:30。

2.3 神经网络时代 (2010 年代)

2.3.1 LSTM 与早期神经语言模型

- **Hochreiter & Schmidhuber (1997)**: 提出长短期记忆网络 (LSTM), 解决了传统 RNN 的梯度消失问题
- **Bengio 等 (2003)**: 发表了第一篇神经语言模型论文。这个模型并非 LSTM, 而是一个前馈网络, 只能处理小范围的上下文

2.3.2 Seq2Seq 与注意力机制

注意力机制的突破

Seq2Seq (序列到序列) 模型大胆地提出: 我们可以把整个句子压缩成一个固定长度的向量。然而, 这个“压缩瓶颈”严重限制了翻译质量。**注意力机制 (Attention)** 的引入解决了这个问题——它允许解码器在生成每个词时, 动态地“关注”输入序列的不同部分, 而不是依赖一个单一的压缩向量。

2.3.3 Transformer 革命

- **Vaswani 等 (2017)**: 提出 Transformer 架构, 完全基于注意力机制, 摒弃了循环结构
- 最初为机器翻译设计, 但很快显示出通用性
- 扩展到 **Mixture of Experts (MoE)** 和模型并行

2.4 预训练时代 (2010 年代末-2020 年代初)

- **ELMo (2018)**: 基于双向 LSTM 的上下文词表示
- **BERT (2018)**: 基于 Transformer 编码器, 通过掩码语言建模进行预训练
- **GPT 系列 (2018-2020)**: 基于 Transformer 解码器, 通过自回归语言建模进行预训练
- **GPT-3 (2020)**: 展示了大尺度预训练的惊人能力 (in-context learning)

2.5 现代大语言模型

Percy Liang 指出, 如今我们对语言模型的要求, “10 年前谁也想象不到”。现代 LLM 不仅能生成流畅文本, 还能执行复杂的智能体编码任务、多轮对话、推理等。

一个关键的反思

尽管模型能力发生了天翻地覆的变化, 但**基本原理并未改变**:

- 仍然依靠 GPU 和内核 (kernels)
- 仍然使用梯度/随机梯度类方法进行优化
- 仍然基于 Transformer 及其注意力机制

变化的是**规格**: 更长的上下文长度、更大的模型规模、对推理效率的更高要求。

2.6 本章小结

从 Shannon 的熵测量到 Transformer 革命，再到今天的 GPT-4 级别模型，语言模型经历了近 70 年的发展。理解这段历史有助于我们把握技术演进的脉络：每一次重大突破（LSTM、注意力、Transformer、预训练）都解决了前一代方法的根本性瓶颈，同时也为下一代问题打开了大门。

3 课程安排与教学理念

3.1 课程大纲

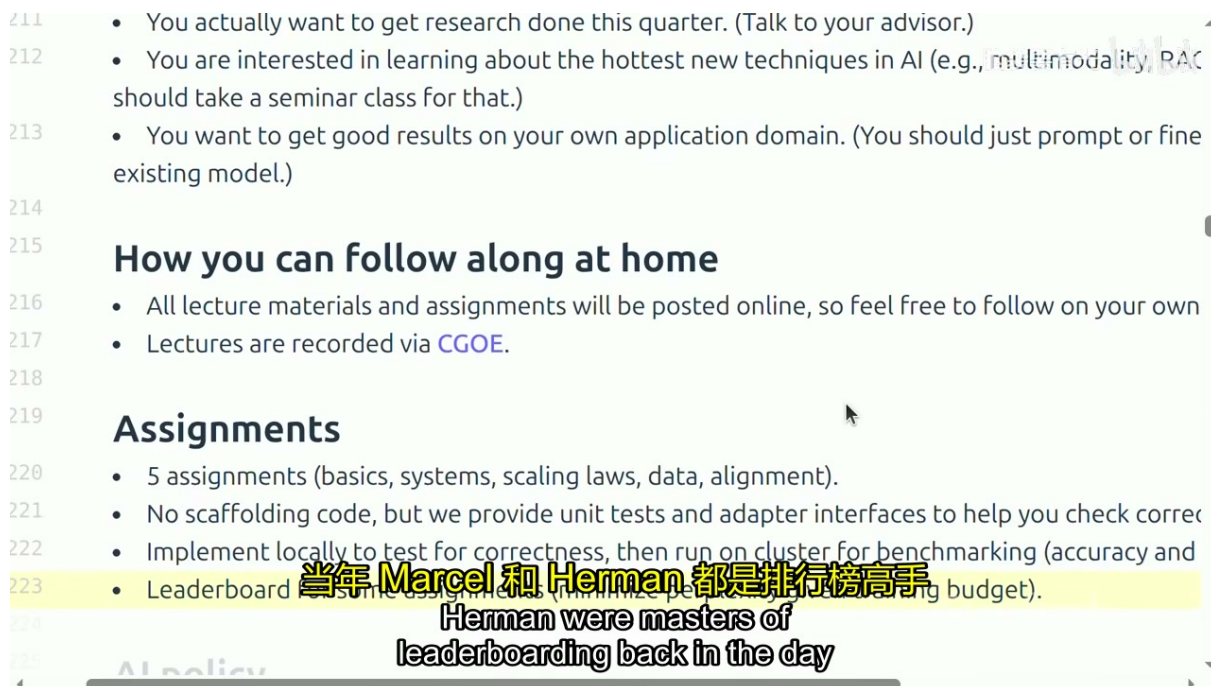


图 4: 课程作业安排与评分标准

课程共包含五次作业，覆盖以下主题：

1. **基础** (basics)：分词、数据加载、简单模型训练
2. **系统** (systems)：GPU 内核、并行化、性能优化
3. **缩放定律** (scaling laws)：小规模实验、超参数迁移、最优配置预测
4. **数据** (data)：数据筛选、去重、数据混合策略
5. **对齐** (alignment)：RLHF、RL 算法、RL 系统

⁰视频画面时间区间：00:24:30-00:25:00。

3.2 作业设计哲学

作业的特点

- **无脚手架代码**：不给你填空的模板，要求从零实现
- **提供单元测试和适配接口**：帮助你验证正确性
- **本地开发 + 集群评测**：先在本地验证正确性，然后在集群上跑 benchmark（精度和效率）
- **排行榜机制**：在给定计算预算下最小化困惑度，鼓励学生竞争最优方案

3.3 AI 辅助学习政策

课程对 AI 工具采取了务实而非禁止的态度，但有明确的规则：

- 承认编程代理已经非常强大，”他们能解决所有作业”
- 但强调：如果直接把作业丢给 AI，”那什么也学不到”
- **强制要求**：如果你使用 AI，**必须使用**课程提供的 `agents.md` 文件（一个精心设计的提示词）。该提示词让 AI 以教学为导向辅助学习，而非直接给出答案
- 目标是借助 AI 来深化理解，而不是替代思考

学术诚信的边界

课程鼓励使用 AI 作为学习工具，但要求学生在最终提交的作品中体现自己的理解。把第一份 PDF 作业直接丢进 Claude Code 固然能得到正确答案，但这违背了课程的教育目标。

3.4 计算资源

本学期的计算资源由 **Modal** 赞助提供。与去年使用 SSH 登录集群不同，今年学生将通过 Modal 平台获取计算额度来完成作业和项目。

3.5 评分标准

- 作业：50%
- 期末项目：40%
- 参与：10%

3.6 本章小结

CS336 通过五次精心设计的作业，系统性地覆盖了从零构建大语言模型的全流程。作业采用”无脚手架 + 排行榜”的模式，既保证了学习的深度，又激发了探索最优方案的动力。课程对 AI 工具采取了引导式开放的态度，强调借助 AI 深化理解而非替代思考。

4 硬件与系统基础

4.1 GPU 架构概览

训练大语言模型的核心硬件是 GPU。讲座以 NVIDIA 的 H100/B200 为例，介绍了关键性能指标。理解这些数字是进行资源核算（resource accounting）的基础。

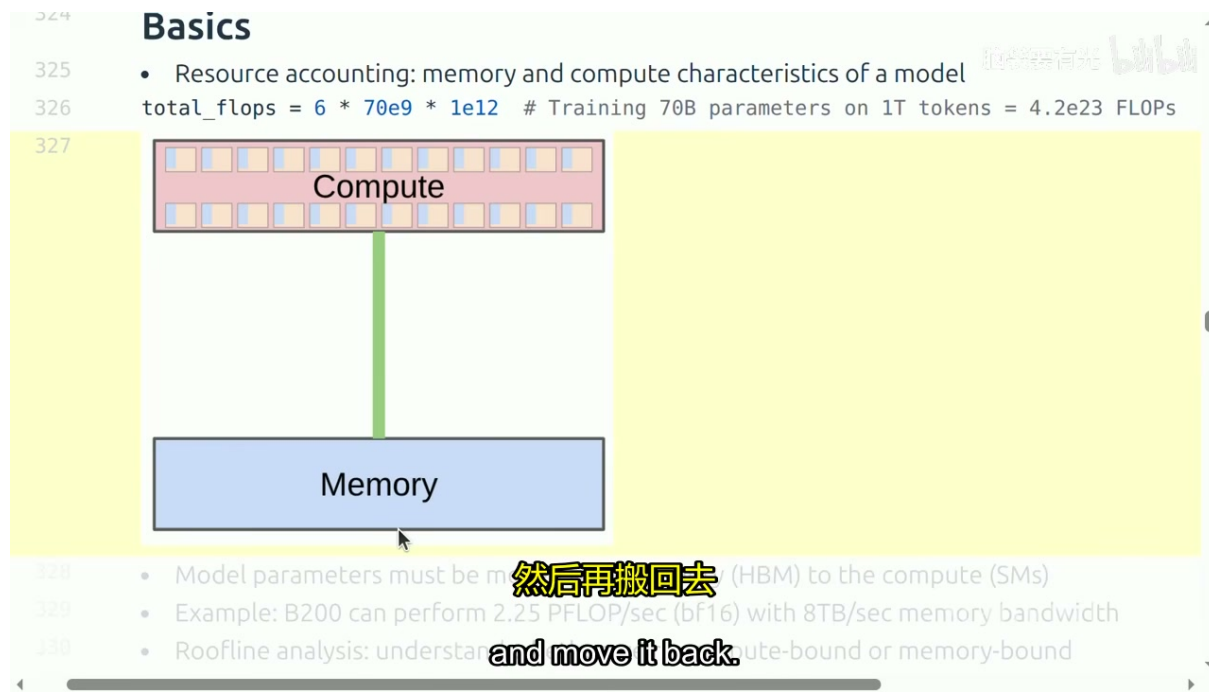


图 5: GPU 计算与内存的基本关系

关键数字

以 NVIDIA B200 为例：

- **BF16 算力**：2.25 PFLOP/sec（每秒 2.25 千万亿次浮点运算）
- **内存带宽**：8 TB/sec（每秒 8 万亿字节）

这两个数字的比值决定了大多数操作的瓶颈所在。

⁰视频画面时间区间：00:37:00–00:37:30。

资源核算的基本公式

训练一个 Transformer 语言模型所需的总计算量可以用一个简单的公式估算：

$$\text{Total FLOPs} = 6 \times N \times D$$

其中：

- N ：模型参数数量
- D ：训练 token 数量

例如，训练一个 70B 参数的模型在 1T（一万亿）token 上，大约需要 $6 \times 70 \times 10^9 \times 10^{12} = 4.2 \times 10^{23}$ 次浮点运算。这个公式是后续所有系统分析和预算规划的基础。

4.2 DGX 系统架构

现代大模型训练通常使用 DGX（Deep Learning GPU eXtreme）系统：

- 单个 DGX 节点搭载 8 个 GPU
- GPU 之间通过 NVLink 高速互联
- 多个 DGX 节点通过 InfiniBand 或以太网连接
- 大规模集群可达 1000+ GPU

4.3 Roofline 分析

Roofline 模型

Roofline 分析是一种判断计算瓶颈的方法。它比较两个量：

- **运算强度**（arithmetic intensity）：每字节数据需要进行多少次运算
- **算力/带宽比**：GPU 的峰值算力与峰值内存带宽之比

如果运算强度低于这个比值，瓶颈在**内存带宽**；如果高于，瓶颈在**算力**。对于大多数 Transformer 操作，**瓶颈通常都在内存带宽**。

4.4 内核与算子融合

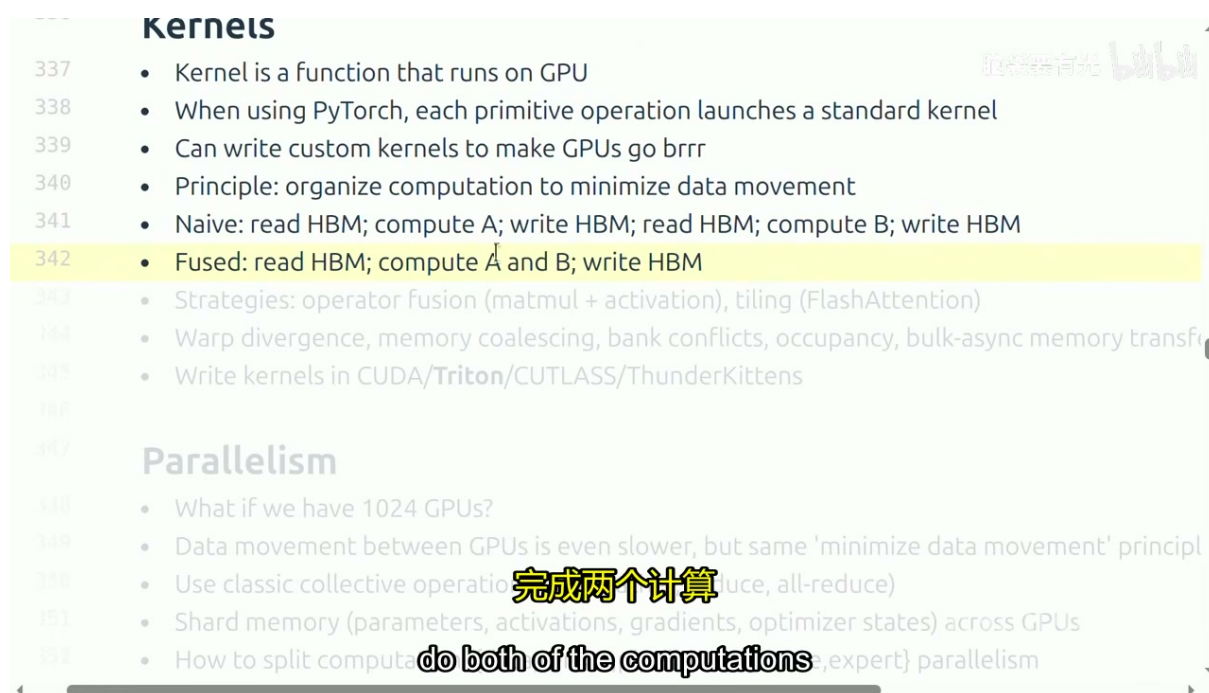


图 6: 内核优化策略：算子融合与分块

内核 (Kernel) 是在 GPU 上运行的函数。当使用 PyTorch 时，每个原语操作实际上都在调用特定的内核。

算子融合的核心思想

从内存中移动数据的代价极其高昂。**算子融合 (Operator Fusion)** 的核心思想是：

- **朴素方式**：读取 HBM → 计算 A → 写回 HBM → 读取 HBM → 计算 B → 写回 HBM (数据搬运两次)
- **融合方式**：读取 HBM → 计算 A 和 B → 写回 HBM (数据搬运一次)

通过减少数据在 GPU 高带宽内存 (HBM) 和计算单元之间的往返，可以显著提升效率。

其他重要的内核优化策略包括：

- **分块 (Tiling)**：如 FlashAttention，将大的注意力计算分解为可在 GPU 共享内存 (SRAM) 中完成的小块
- **Warp 发散控制、内存合并访问、Bank 冲突避免、Occupancy 优化**
- 编写自定义内核的工具：CUDA、**Triton**、CUTLASS、ThunderKittens

4.5 并行化简介

当拥有 1024 个 GPU 时，关键问题变为：

⁰ 视频画面时间区间：00:39:30–00:40:00。

- GPU 之间的数据移动比 GPU 内部更慢，但”最小化数据移动”的原则不变
- 使用经典集合通信操作：broadcast、reduce、all-reduce
- 内存分片：将参数、激活值、梯度、优化器状态分散到多个 GPU
- 并行策略：数据并行、张量并行、流水线并行、专家并行（MoE）

4.6 本章小结

理解硬件是高效训练大模型的前提。GPU 的算力和内存带宽构成了基本约束，Roofline 分析帮助我们定位瓶颈。算子融合和分块是提升内核效率的核心技术，而并行化策略则让我们能够将计算扩展到数千个 GPU。所有这些技术的共同目标都是**最小化数据移动**——无论是在单个 GPU 内部，还是在集群中的 GPU 之间。

5 分词技术

5.1 为什么需要分词

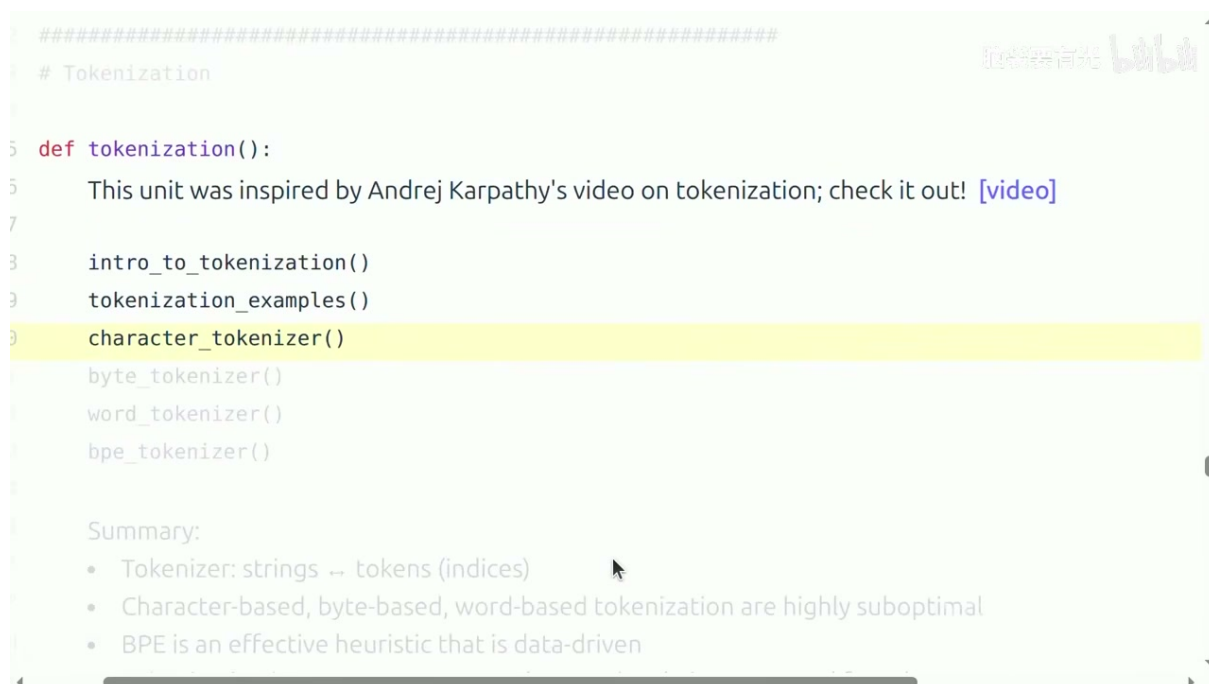


图 7: 分词单元的整体结构

分词（Tokenization）是语言模型 pipeline 的第一环，其本质是一个双射：

$$\text{字符串} \longleftrightarrow \text{token 序列 (索引)} \quad (1)$$

⁰视频画面时间区间：01:10:00–01:10:30。

为什么不能直接用原始字节？

直接在原始字节序列上训练模型看似优雅，但存在根本性问题：

- 在文本、视频或 DNA 序列中，**单个字节的噪声比信号还大**
- 必须进行抽象，提升到模型能够有效建模的层次
- 不同字节不应被同等对待（例如空格和字母的信息含量不同）

因此，将原始文本映射到更高层次的 token 表示是必要的预处理步骤。

5.2 分词策略的比较

讲座讨论了三种基本的分词策略，并指出了各自的优缺点：

表 1: 不同分词策略的比较

策略	词汇表大小	序列长度	主要问题
字符级	很小 (100)	很长	序列过长，计算开销大
词级	很大 (100k+)	较短	OOV (未登录词)、语言差异
字节级	256	很长	计算低效，信息密度低

各策略的详细分析

字符级分词：词汇表极小（仅包含所有可能的字符），但序列长度会膨胀数倍。例如一个英文单词平均需要 5-6 个字符表示，导致 Transformer 需要处理更长的序列。

词级分词：序列短，但词汇表爆炸式增长。不同语言的词形态差异巨大（例如中文没有空格分词），且存在严重的 OOV 问题——新词、专有名词、拼写错误都无法处理。

字节级分词：优雅且通用（所有文本都可以表示为 UTF-8 字节序列），但序列过长导致计算效率低下。对于现代 Transformer 架构而言，这会带来不可接受的计算开销。

压缩比：分词效率的核心指标

压缩比 (compression ratio) 是衡量分词器效率的关键指标，定义为：

$$\text{压缩比} = \frac{\text{原始字节数}}{\text{token 数量}}$$

例如，一个 20 字节的字符串如果被编码为 8 个 token，压缩比就是 $20/8 = 2.5$ 。**压缩比越高，序列越短**，这对于 Transformer 尤为重要——因为注意力机制的复杂度与序列长度的平方成正比，更短的序列意味着更快的训练和推理。

分词器的具体陷阱

Percy Liang 指出，当前的分词器存在许多令人头疼的 quirks，这也是“所有人都想摆脱分词”的原因之一：

- **前导空格问题**：同一个单词带前导空格和不带前导空格会变成完全不同的 token。例如，GPT-2 中“hello”和“ hello”（带前导空格）是两个不相关的索引
- **数字分块不一致**：不同分词器对数字的处理方式截然不同——有的按单个数字切分，有的按固定长度 chunk，有的保留完整数字
- **相同字符串的不同表示**：由于 BPE 的贪心合并策略，同一个子串在不同上下文中可能被切分为不同的 token 序列

这些缺陷使得分词器成为模型 pipeline 中一个“不得不存在但人人想换掉”的组件。

5.3 BPE 算法详解

BPE：数据驱动的有效启发式

BPE (Byte Pair Encoding) 是目前最广泛使用的分词算法。它由 Sennrich 等 (2016) 为神经机器翻译提出，后来被 OpenAI 的 GPT 系列采用并发扬光大。BPE 的核心思想是：从字节序列出发，通过迭代合并最频繁的相邻 token 对，逐步构建词汇表。

5.3.1 算法步骤

1. **初始化**：将输入文本转换为字节序列（每个字节的值为 0-255）， $\text{vocab} = 256$
2. **统计**：扫描整个语料，统计所有相邻 token 对的出现频率
3. **合并**：选择频率最高的 token 对，将其合并为一个新 token，加入词汇表
4. **重复**：重复步骤 2-3，直到词汇表达到预设大小（例如 50,000）

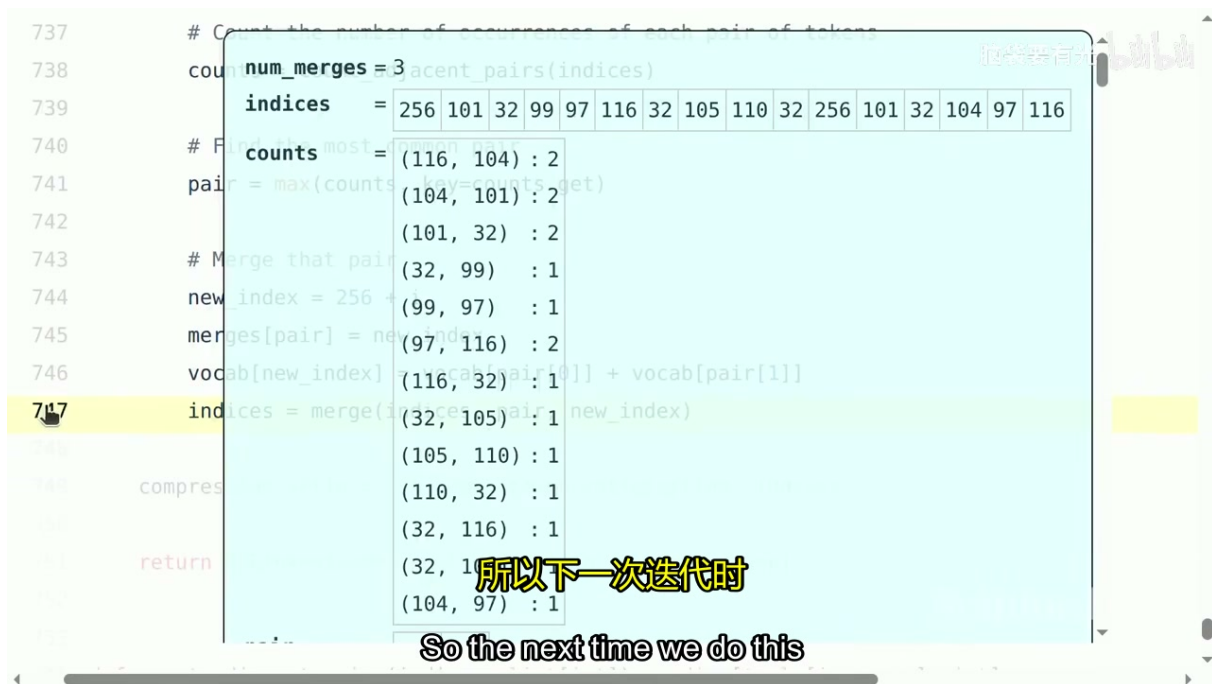


图 8: BPE 算法迭代合并过程的演示

5.3.2 代码实现

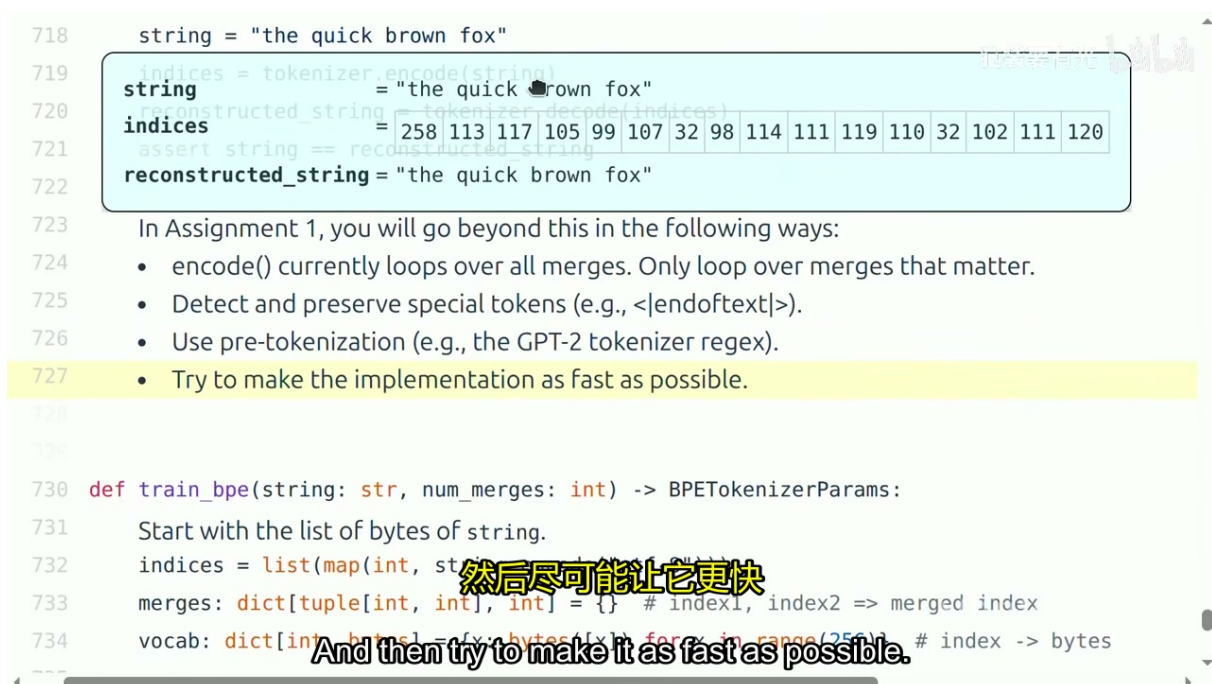


图 9: BPE 编码和解码的代码实现

```
1 def train_bpe(string: str, num_merges: int) -> BPETokenizerParams:
```

⁰视频画面时间区间: 01:15:00-01:15:30。

⁰视频画面时间区间: 01:17:00-01:17:30。

```

2  # 从字节序列开始
3  indices = list(map(int, string.encode("utf-8")))
4  merges: dict[tuple[int, int], int] = {} # index1, index2 -> merged index
5  vocab: dict[int, bytes] = {i: bytes([i]) for i in range(256)}
6
7  for _ in range(num_merges):
8      # 统计所有相邻 token 对的出现次数
9      counts = count_adjacent_pairs(indices)
10     # 选择频率最高的 pair
11     pair = max(counts, key=counts.get)
12     # 分配新索引
13     new_index = 256 + len(merges)
14     merges[pair] = new_index
15     vocab[new_index] = vocab[pair[0]] + vocab[pair[1]]
16     # 在序列中执行合并
17     indices = merge(indices, pair, new_index)
18
19  return BPETokenizerParams(merges=merges, vocab=vocab)

```

Listing 1: BPE 算法的核心逻辑

编码 (encode) 时, 从字节序列开始, 依次应用所有合并规则; 解码 (decode) 时, 将 token 索引映射回字节序列, 再用 UTF-8 解码为字符串。

BPE 的微妙之处

BPE 的一个关键特性是**贪心合并**——它总是优先合并频率最高的 pair, 但这并不保证全局最优的分词方案。此外, BPE 的训练是**确定性的** (给定相同的数据和合并次数, 总是得到相同的词汇表), 但编码时可能会遇到训练时未见过的字符组合, 需要回退到字节级表示。

5.3.3 编码性能与优化

上面展示的 `train_bpe` 实现虽然清晰易懂, 但有一个严重的性能问题: **编码 (encode) 过程非常慢**。朴素的实现需要对输入序列迭代遍历所有合并规则, 时间复杂度与词汇表大小和序列长度成正比。

第一次作业的优化目标

在课程第一次作业中, 学生需要对上述 BPE 实现进行以下优化:

1. **索引优化**: 只遍历“相关的”合并规则, 而非全部
2. **特殊 token 处理**: 检测并保留特殊 token (如 `<|endoftext|>`)
3. **预分词**: 使用预分词策略 (如 GPT-2 的正则表达式) 将文本切分为 chunk, 再对每个 chunk 分别编码
4. **速度优化**: 尽可能让实现更快

这些优化将 naive 的 $O(V \cdot L)$ 编码复杂度降低到接近 $O(L)$, 是理解分词器工业实现的关键一步。

真实分词器的额外细节

实际生产中的分词器还包含以下概念上并不复杂但对构建现代分词器很重要的细节：

- **分块编码**：真实 tokenizer 不会一次性处理整个字符串，而是先将文本切分为多个 chunk，再并行或逐块编码
- **特殊 token**：如 `<|endoftext|>`、`<|padding|>` 等，用于标记序列边界或填充
- **预分词正则**：在 BPE 合并之前，先用正则表达式将文本切分为初步的单元（如 GPT-2 使用的特定正则）

5.4 各种 Tokenizer 的比较

不同的模型采用了不同的分词策略：

- **GPT-2**：使用 BPE，词汇表大小 50,257，使用特定的预分词正则表达式
- **GPT-4**：改进的 BPE，更好地处理多语言和代码
- **Llama**：使用 SentencePiece 实现的 BPE，词汇表更大（32K-128K）
- **其他**：WordPiece（BERT）、Unigram（T5）等变体

5.5 分词的未来方向

Percy Liang 在讲座结尾提出了几个关于分词未来发展的思考：

分词的理想特性

任何替代分词的端到端方案，都必须满足以下特性：

1. **抽象性**：模型需要在某种抽象表示上运算，而非原始字节
2. **可变性**：数据块的大小应该可以变化（自适应计算）
3. **非均匀性**：不同字节不应被同等对待

Percy 还幽默地提到：“也许明年我就不用再讲分词了”——暗示端到端学习分词的方法（如直接学习字节级表示或动态分组）可能在不久的将来取代 BPE。但今年，BPE 仍然是工业界和学术界的主流选择。

5.6 本章小结

分词是语言模型 pipeline 的第一环，虽小但关键。BPE 通过数据驱动的合并策略，在词汇表大小和序列长度之间取得了良好的平衡。尽管字符级、词级、字节级分词各有优劣，但 BPE 凭借其简洁性和有效性成为了事实标准。未来，随着自适应计算和端到端学习方法的发展，固定词汇表的分词方式可能会被更灵活的方案所取代。

6 总结与延伸

6.1 讲座核心回顾

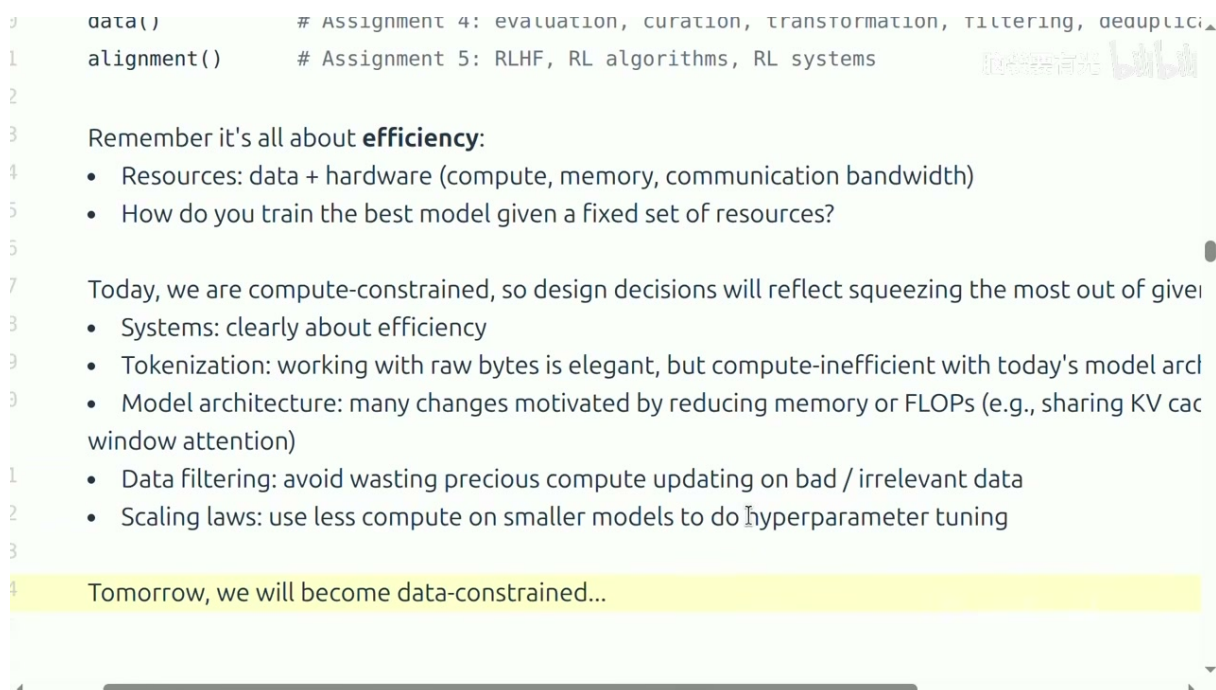


图 10: 课程核心理念：效率

本讲座作为 CS336 的导论，围绕一个核心主题展开：**效率** (efficiency)。具体而言：

效率的核心公式

资源 = 数据 + 硬件（算力、内存、通信带宽）

核心问题：**如何在固定资源约束下训练出最好的模型？**

讲座从历史、系统、分词三个维度阐述了这个主题：

1. **历史维度**：从 N-gram 到 Transformer，从 ELMo 到 GPT-4，技术范式不断演进，但核心原理（注意力、优化、内核）保持不变
2. **系统维度**：理解 GPU 架构、内存带宽瓶颈、算子融合和并行化策略，是高效训练的前提
3. **分词维度**：BPE 作为数据驱动的启发式方法，是连接原始文本与模型输入的桥梁

6.2 当前与未来的约束

Percy Liang 提出了一个重要观点：

⁰视频画面时间区间：01:05:00–01:05:30。

从算力受限到数据受限

今天，我们是**算力受限**（compute-constrained）的——系统设计和算法优化的目标是最大化利用有限的计算资源。

明天，我们可能会变成**数据受限**（data-constrained）的——当计算资源持续增长而高质量文本数据枯竭时，如何获取、筛选、合成数据将成为新的瓶颈。

这一观点深刻影响了课程的设计：五次作业中有两次（scaling laws 和 data）直接围绕资源约束展开。

6.3 进一步思考

1. **可执行讲座的价值**：将教学内容组织为可运行的代码，不仅消除了理论与实践的鸿沟，还让学生能够通过修改参数、观察输出来建立直觉。这种形式值得在更多技术课程中推广。
2. **BPE 的局限性**：BPE 虽然是当前的最佳实践，但它的贪心合并策略并不保证最优，且固定词汇表的设计限制了模型的灵活性。字节级模型（如 ByT5）和动态分词（如 Mamba 的潜在方法）可能代表未来方向。
3. **效率 vs 能力**：课程强调效率，但不应忽视“能力”（capability）的维度。有时候，增加 10% 的计算可以带来 50% 的能力提升，这种权衡需要在实际项目中仔细评估。
4. **AI 辅助学习的边界**：课程对 AI 工具的开放态度是一种进步。关键在于设计好的“脚手架”（如 agents.md），引导学生正确使用 AI，而非简单禁止或放任。

6.4 拓展阅读

- **BPE 原始论文**：Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*.
- **Andrej Karpathy 的分词视频**：讲座中特别推荐的补充材料，从实现角度深入讲解 BPE
- **课程网站**： <https://stanford-cs336.github.io/spring2026/> ——所有讲义、作业、代码均可在线获取